

Clase 1: Arquitectura Web y Primer Servidor con Python

Bienvenido a **Diseño Web II**. En esta clase pasaremos de crear páginas estáticas a construir aplicaciones web dinámicas que "piensan" y responden desde un servidor.

1. Introducción Teórica: ¿Qué es el Backend?

Cliente vs. Servidor

- **El Cliente (Frontend):** Es el navegador (Chrome, Safari). Su trabajo es mostrar la interfaz y reaccionar a los clics del usuario.
- **El Servidor (Backend):** Es una computadora (o programa) que espera peticiones, procesa datos (consulta bases de datos, verifica contraseñas) y envía una respuesta.

¿Por qué Python para la Web?

Python es uno de los lenguajes más populares del mundo por su sintaxis clara (parecida al inglés) y su enorme ecosistema. Frameworks como **Flask** nos permiten levantar un servidor en menos de 10 líneas de código. Trabajaremos con el intérprete 3.11.1

<https://www.python.org/downloads/release/python-3111/>

2. Configuración del Entorno Profesional

Para trabajar de forma profesional, no instalamos librerías globalmente en la computadora. Usamos **Entornos Virtuales (Venv)**. Imagina que cada proyecto es una caja cerrada con sus propias herramientas.

En primer lugar deben tener instalado VSC <https://code.visualstudio.com/> y el intérprete acoplado antes de continuar

Paso a paso en la Terminal cmd:

Crear carpeta del proyecto:

```
mkdir mi_primer_servidor  
cd mi_primer_servidor
```

Crear el entorno virtual:

```
python -m venv venv
```

Activar el entorno en VSC:

- **Windows:** venv\Scripts\activate
- **Mac/Linux:** source venv/bin/activate
(Verás un prefijo (venv) en tu terminal).

Instalar Flask:

pip install Flask sobre el ejecutable (app.py)

Paso a paso tradicional:**Crear carpeta del proyecto:**

mi_primer_servidor

Crear el entorno virtual en VSC:

pip install pipenv

pipenv shell

cambiar de intérprete (VENV)

Instalar Flask:

pipenv install Flask sobre el ejecutable (app.py)

3. Estructura del Proyecto

Crea los siguientes archivos y carpetas dentro de mi_primer_servidor:

```
mi_primer_servidor/  
├── venv/  
├── app.py      <-- Lógica del servidor  
├── templates/  <-- Carpeta para archivos HTML  
└── index.html  <-- Nuestra plantilla
```

4. El Código: Hola Mundo Dinámico

app.py (El Servidor)

```
from flask import Flask, jsonify, render_template
```

```
app = Flask(__name__)
```

```
# Ruta que devuelve HTML
```

```
@app.route('/')
```

```
def home():
```

```
    return render_template('index.html')
```

```
# Ruta que devuelve JSON (Formato de datos para APIs)
```

```
@app.route('/api/hola')
```

```
def hola_json():
    return jsonify({
        "status": "success",
        "mensaje": "¡Hola Mundo desde el Backend!",
        "clase": "Diseño Web II"
    })

if __name__ == '__main__':
    app.run(debug=True)
```

templates/index.html (La Vista)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Primer Servidor</title>
    <script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <div class="bg-white p-8 rounded-lg shadow-md text-center">
        <h1 class="text-3xl font-bold text-blue-600 mb-4">¡Servidor Activo!</h1>
        <p class="text-gray-600">Esta página está siendo servida por Python y Flask.</p>
        <div class="mt-6">
            <a href="/api/hola" class="text-sm text-indigo-500 underline">Ver respuesta JSON</a>
        </div>
    </div>
</body>
</html>
```

5. Ejercicio Práctico

1. **Lanzar el servidor:** En la terminal escribe `python app.py`.
2. **Visitar la web:** Abre tu navegador en `http://127.0.0.1:5000`.
3. **Reto:** Cambia el mensaje del diccionario en la función `hola_json` en `app.py`, guarda el archivo y refresca el navegador. ¿Qué sucede?

6. Ejercicio creativo

Cada uno debe crear una ruta `/mi-perfil` que devuelva su nombre, su edad y sus tres redes sociales favoritas en formato JSON

Conceptos Clave de Hoy

- **Protocolo HTTP:** El lenguaje de peticiones y respuestas.
- **Rutas (@app.route):** Las direcciones URL de nuestra aplicación.
- **JSON:** El formato estándar para intercambiar datos en la web moderna.